

Study of 8086 Microprocessor Architecture

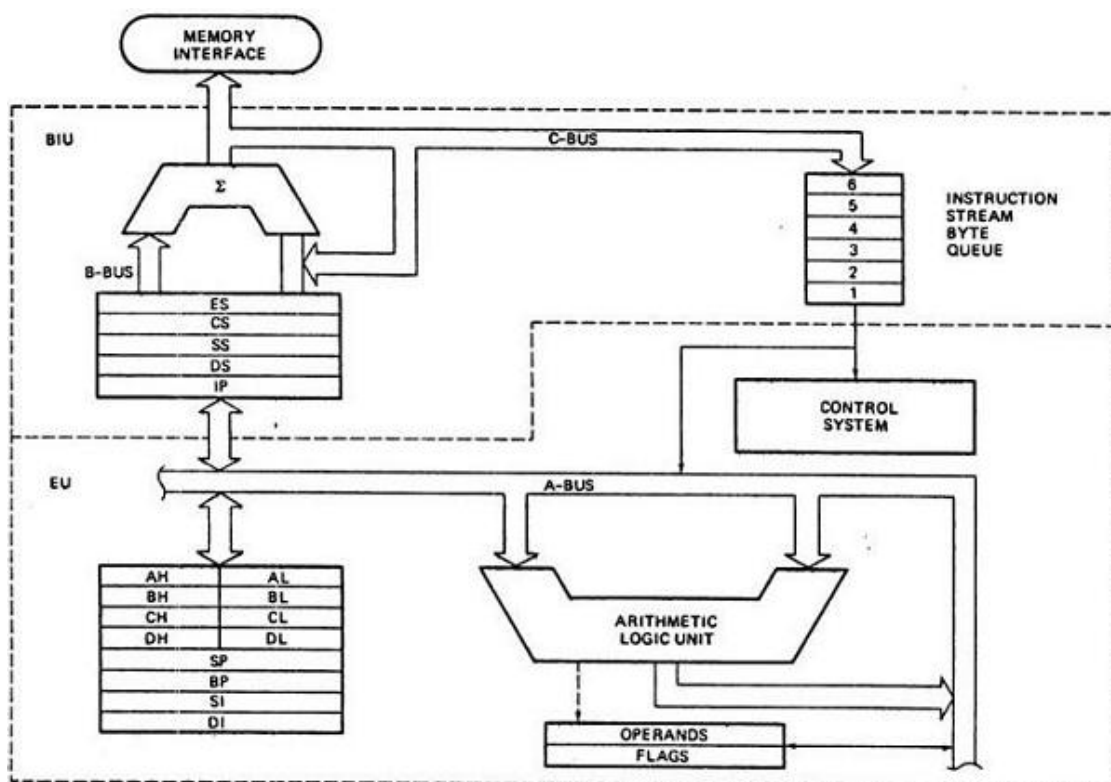
1. Architecture:

The Intel 8086 is a 16-bit microprocessor introduced by Intel in 1978. It is the foundation of the x86 family of processors and was designed to execute complex instructions efficiently.

Main Features:

- 16-bit processor with 20-bit address bus → can access **1 MB memory** (2^{20} bytes)
- Has **segmented memory** architecture
- Contains **Arithmetic Logic Unit (ALU)** for arithmetic and logical operations
- 14 registers (general, segment, and pointer/index)
- **Clock speed** typically between 5–10 MHz
- Supports **multiprogramming** and **pipelining** (instruction queue)

Block Diagram:



How it works:

- The BIU fetches the next instruction while EU executes the current one — providing **pipelined execution**.
- This improves performance by overlapping fetch and execute cycles

2. Register Organization:

8086 has several types of registers categorized as:

I. General Purpose Registers (GPR):

Register	Purpose	Description
AX	Accumulator	Used in arithmetic, logic, and data transfer
BX	Base Register	Used in indexed addressing
CX	Count Register	Used in loops and shifts
DX	Data Register	Used in multiplication/division and I/O operations

II. Segment Registers:

Register	Purpose
CS	Code Segment – holds program code
DS	Data Segment – holds variables/data
SS	Stack Segment – manages stack operations
ES	Extra Segment – used for string operations

III. Pointer and Index Registers:

Register	Purpose
SP	Stack Pointer
BP	Base Pointer
SI	Source Index
DI	Destination Index
IP	Instruction Pointer (points to next instruction)

IV. Flag Register:

- Contains **status flags** (like Carry, Zero, Sign, Overflow) and **control flags**.
- Used to indicate the outcome of operations.

3. Memory Segmentation in 8086:

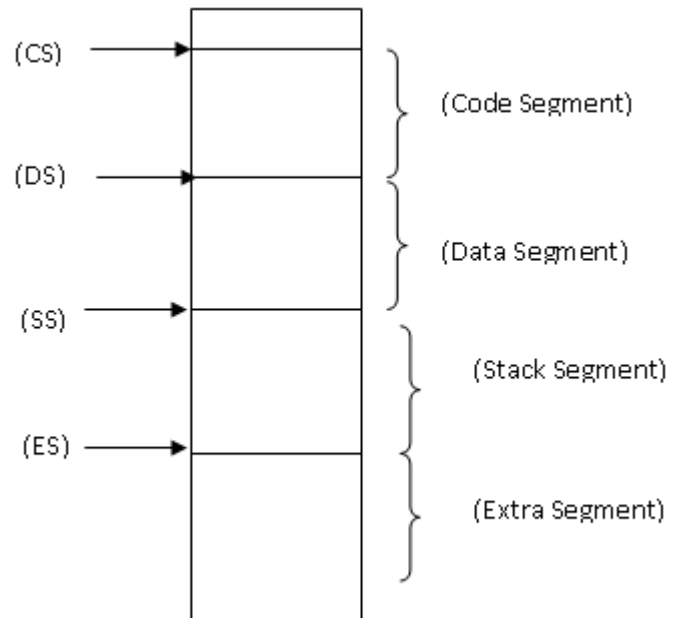
The 8086 uses a segmented memory model to efficiently access its 1 MB address space. Memory is divided into four main segments:

1. **Code Segment:** Contains executable program instructions.
2. **Data Segment:** Holds variables, data, and constants.
3. **Stack Segment:** Stores temporary data, return addresses, and handles function calls.
4. **Extra Segment:** Used for additional data and string operations.

Each segment is 64 KB in size. The physical memory address is calculated using both the segment and offset addresses with the formula:

$$\text{Physical Address} = (\text{Segment} \times 10\text{H}) + \text{Offset}$$

For example, if CS = 2000H and IP = 1234H, then the physical address = 2000H × 10H + 1234H = 21234H.



4. Example Programs (Assembly Language)

a. Hello World Program

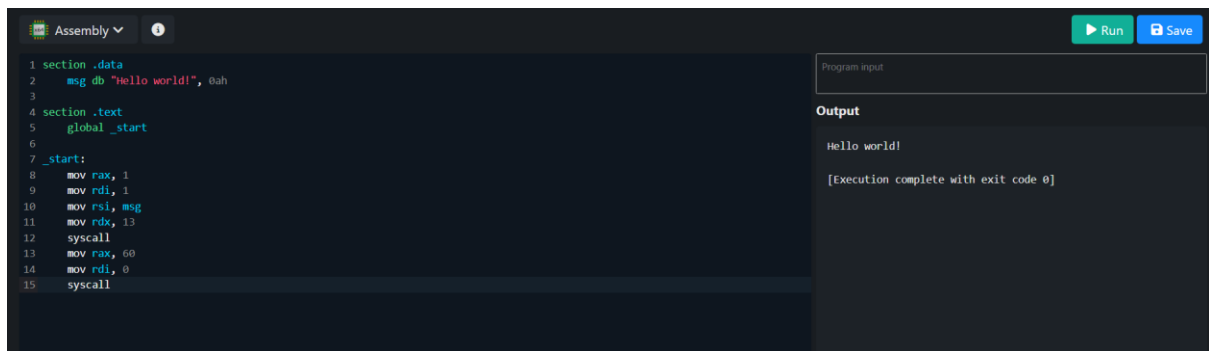
Code:

```
section .data
    msg db "Hello world!", 0ah
```

```
section .text
    global _start
```

```
_start:
    mov rax, 1
    mov rdi, 1
    mov rsi, msg
    mov rdx, 13
    syscall
    mov rax, 60
    mov rdi, 0
    syscall
```

Output:



The screenshot shows an assembly code editor with a dark theme. The code is written in x86-64 assembly and is titled 'Assembly'. It defines a data section with a message 'Hello world!', a text section with a global start label, and a main function that prints the message using the syscalls. The output panel on the right shows the program's execution, displaying 'Hello world!' and '[Execution complete with exit code 0]'. The code is as follows:

```
1 section .data
2     msg db "Hello world!", 0ah
3
4 section .text
5     global _start
6
7 _start:
8     mov rax, 1
9     mov rdi, 1
10    mov rsi, msg
11    mov rdx, 13
12    syscall
13    mov rax, 60
14    mov rdi, 0
15    syscall
```

Program input:

Output:

Hello world!

[Execution complete with exit code 0]

b. Addition of Two Number

Code:

```
section .data
    num1 db 3
    num2 db 5
    result db 0
```

```
section .text
    global _start
```

```
_start:
    mov al, [num1]
    add al, [num2]
    add al, '0'
    mov [result], al
```

```
    mov rax, 1
    mov rdi, 1
    mov rsi, result
    mov rdx, 1
    syscall
```

```
    mov rax, 60
    xor rdi, rdi
    syscall
```

Output:

Assembly

```
1 section .data
2     num1 db 3
3     num2 db 5
4     result db 0
5
6 section .text
7     global _start
8
9 _start:
10    mov al, [num1]
11    add al, [num2]
12    add al, 0
13    mov [result], al
14
15    mov rax, 1
16    mov rdi, 1
17    mov rsi, result
18    mov rdx, 1
19    syscall
20
21    mov rax, 60
22    xor rdi, rdi
23    syscall
24
```

Run

Save

Program input

Output

8
[Execution complete with exit code 0]